پروژه‌ی Paradiso — بک‌اند (NestJS)

راهنمای Redis

یکپارچه‌سازی و راه‌اندازی کش پاسخ‌های TMDb

و توضیح ماژول سراسری (Global Module)

۹ آبان ۱۴۰۵ (۳۱ اکتبر ۲۰۲۶) · نوشته‌شده برای توسعه‌دهنده‌ای که با بک‌اند تازه آشنا می‌شود

فهرست مطالب

۱. Redis چیست و چرا در این پروژه؟

۲. تصویر کلی: Redis کجا قرار می‌گیرد؟

۳. راه‌اندازی گام‌به‌گام

۴. فایل‌هایی که اضافه یا تغییر کردند

۵. سرویس `CacheService` از درون

۶. اتصال به `MovieDb.serviec.ts` و اثر آن

۷. نتیجه‌ی اندازه‌گیری روی Render

۸. نکات و محدودیت‌ها

۹. ماژول سراسری: Redis چگونه Global تعریف شده است؟

۱. Redis چیست و چرا در این پروژه؟

Redis یک پایگاه‌داده‌ی «کلید ← مقدار» است که داده‌ها را در حافظه‌ی رم نگه می‌دارد؛ برای همین بسیار سریع است. اما چون حافظه‌ی موقت است، قرار نیست جای پایگاه‌داده‌ی اصلی را بگیرد.

در Paradiso دیتابیس اصلی **Postgres روی Neon** است و منبع حقیقت و دائمیِ داده‌ها محسوب می‌شود. Redis فقط یک **«میز کار موقت»** کنار آن است: چیزهایی که بارها خوانده می‌شوند و تغییرشان کند است را روی میز می‌گذاریم تا هر بار سراغ قفسه‌ی دور نرویم.

مشکلی که حل کردیم

سرویس `MovieDbService` برای هر فیلم، بازیگر یا جستجو از API سایت TMDb داده می‌گیرد. این کار سه مشکل داشت:

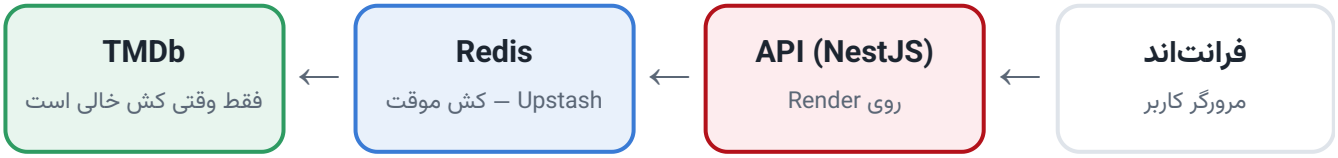
- **کند است:** هر درخواست حدود ۵۰۰ میلی‌ثانیه طول می‌کشد (و گاهی ۲ تا ۴ ثانیه).
- **محدودیت نرخ دارد:** TMDb اگر زیاد درخواست بزنیم خطای ۴۲۹ می‌دهد.
- **جستجو گران است:** برای فیلتر کردن فیلم‌های نامناسب، هر جستجو تا ۲۰ درخواست اضافه (کلیدواژه‌ها) هم می‌زد؛ یعنی حدود ۱۹ تا ۲۱ تماس با TMDb برای یک جستجو.

با Redis، پاسخ‌های TMDb را برای مدتی نگه می‌داریم و دفعه‌ی بعد بدون تماس با TMDb جواب می‌دهیم.

> **قانون طلایی این پیاده‌سازی**
>
> Redis فقط یک **شتاب‌دهنده** است، نه ضرورت. اگر Redis تنظیم نشده باشد، خاموش باشد یا خطا بدهد، برنامه مثل قبل مستقیم از TMDb می‌خواند و خطایی به کاربر نمی‌رسد. هیچ داده‌ای فقط در Redis نیست و migration هم لازم نبود.

۲. تصویر کلی: Redis کجا قرار می‌گیرد؟

Redis یک **سرور جداگانه** است، نه کتابخانه‌ای داخل برنامه. برنامه‌ی Nest از راه شبکه به آن وصل می‌شود، درست مثل اتصال به Neon.



مسیر یک درخواست برای **اطلاعات یک فیلم**:

۱. فرانت‌اند اطلاعات فیلم را می‌خواهد.

۲. سرویس اول از Redis می‌پرسد: «این فیلم را داری؟»

۳. **اگر داشت (Hit):** همان لحظه جواب می‌دهد؛ TMDb صدا زده نمی‌شود.

۴. **اگر نداشت (Miss):** از TMDb می‌گیرد، در Redis ذخیره می‌کند (با زمان انقضا) و جواب می‌دهد.

Neon (دیتابیس اصلی) در این مسیر دخالتی ندارد؛ Redis فقط جلوی TMDb نشسته است.

۳. راه‌اندازی گام‌به‌گام

۱ **ساخت دیتابیس در Upstash.** در سایت Upstash یک دیتابیس Redis با پلن رایگان بسازید. **منطقه (Region) را نزدیک سرور Render انتخاب کنید**؛ بعد از ساخت قابل تغییر نیست.

۲ **برداشتن آدرس TCP.** Upstash دو آدرس می‌دهد. آدرسی که با `rediss://` شروع می‌شود (TCP با رمزنگاری TLS) را بردارید، نه آدرس REST را.

۳ **گذاشتن آدرس در متغیر محیطی.** آن را در فایل `.env.development` و در بخش Environment پنل Render با نام `REDIS_URL` بگذارید.

۴ **نصب بسته.** کلاینت Redis برای Node:

```
npm install ioredis
```

۵ **خواندن تنظیم در کانفیگ.** در `src/config/app.config.ts`:

```
// Optional: without it the app runs uncached.
redisUrl: process.env.REDIS_URL,
```

۶ **اعتبارسنجی (اختیاری).** در `src/config/environment.validation.ts` با Joi فقط فرمت آدرس بررسی می‌شود و متغیر *الزامی نیست*:

```
REDIS_URL: Joi.string().uri({ scheme: ['redis', 'rediss'] }),
```

۷ **ثبت ماژول.** `CacheModule` یک‌بار در `AppModule` اضافه می‌شود (توضیح کامل در بخش آخر):

```
imports: [
  ...
  ScheduleModule.forRoot(),
  CacheModule,
  ...
]
```

تفاوت TCP و HTTP در یک نگاه

	`rediss://` — TCP	`https://` — REST
روش کار	یک اتصال باز می‌ماند و بارها استفاده می‌شود	برای هر دستور یک درخواست HTTP جدید
سرعت	سریع‌تر	کندتر
مناسب برای	سرور همیشه‌روشن مثل Nest روی Render ✓	محیط‌های serverless و edge

> **امنیت آدرس اتصال**
>
> `REDIS_URL` شامل رمز است. آن را در چت، گیت یا لاگ نگذارید. فایل `.env.development` در گیت نادیده گرفته می‌شود. اگر آدرس جایی فاش شد، در Upstash رمز را تعویض (rotate) کنید.

۴. فایل‌هایی که اضافه یا تغییر کردند

فایل	نقش
`src/cache/cache.service.ts`	**جدید.** اتصال به Redis و متدهای `get`، `set`، `delete` و `remember`
`src/cache/cache.module.ts`	**جدید.** ماژول سراسری که `CacheService` را در اختیار همه می‌گذارد
`src/cache/cache.service.spec.ts`	**جدید.** ۸ تست با Redis جعلی (بدون اتصال واقعی)
`src/movies/providers/MovieDb.serviec.ts`	کش روی تماس‌های TMDb اضافه شد
`src/app.module.ts`	`CacheModule` ثبت شد

`src/config/app.config.ts`	خواندن `REDIS_URL`
`src/config/environment.validation.ts`	اعتبارسنجی اختیاریِ `REDIS_URL`
`scripts/benchmark-tmdb-cache.cjs`	اسکریپت اندازه‌گیری سرعت (قبل و بعد)
`docs/redis-cache.md`	مستند فنیِ این قابلیت و جدول‌های اندازه‌گیری

۵. سرویس CacheService از درون

اتصال امن و سریع‌شکست

سازنده‌ی سرویس اگر `REDIS_URL` نباشد یا محیط `test` باشد، اصلاً وصل نمی‌شود (حالت «عبور مستقیم»). در غیر این صورت با این تنظیمات وصل می‌شود:

```
this.redis = new Redis(url, {
  connectTimeout: 10_000,     // connecting may be slow
  commandTimeout: 500,        // a slow Redis must not slow requests
  maxRetriesPerRequest: 1,
  enableOfflineQueue: false,  // fail at once while disconnected
  retryStrategy: (attempt) => Math.min(attempt * 500, 10_000),
});
```

- **commandTimeout = ۵۰۰ms:** اگر Redis دیر جواب دهد، آن دستور «Miss» حساب می‌شود و برنامه سراغ TMDb می‌رود.
- **enableOfflineQueue = false:** هنگام قطع بودن اتصال، دستورها صف نمی‌شوند و بلافاصله شکست می‌خورند (و سرویس خودش را بازیابی می‌کند).
- **در تست خاموش است:** مجموعه‌ی تست دیتابیس فایل `.env.development` را بارگذاری می‌کند؛ بدون این محافظ، تست‌ها به Redis واقعی می‌نوشتند.

متدها

متد	کار
`get(key)`	مقدار ذخیره‌شده را برمی‌گرداند؛ در Miss یا هر مشکل Redis مقدار `undefined` می‌دهد (هرگز خطا پرتاب نمی‌کند)
`set(key, value, ttl)`	مقدار را با زمان انقضا (ثانیه) می‌نویسد؛ هرگز خطا پرتاب نمی‌کند
`delete(key)`	یک کلید را حذف می‌کند
`remember(key, ttl, load)`	**متد اصلی:** اگر در کش بود همان را بده؛ وگرنه `load()` را اجرا کن، نتیجه را ذخیره کن و برگردان

منطق متد `remember`

```
remember(key, ttlSeconds, load) {
  const running = this.loading.get(key);
  if (running) return running;          // share one load

  const promise = (async () => {
    const cached = await this.get(key);
    if (cached !== undefined) return cached;   // HIT

    const value = await load();                // MISS: ask TMDb
    void this.set(key, value, ttlSeconds);     // write, do not wait
    return value;
  })().finally(() => this.loading.delete(key));

  this.loading.set(key, promise);
  return promise;
}
```

نکته‌های مهم این کد:

- **اشتراک تماس هم‌زمان (single-flight):** اگر ده کاربر هم‌زمان یک فیلم را بخواهند، فقط یک تماس با TMDb انجام می‌شود و همه‌ی ده نفر از همان نتیجه استفاده می‌کنند.
- **خطا ذخیره نمی‌شود:** اگر `load()` خطا بدهد (مثلاً ۴۰۴ یا قطعی TMDb)، چیزی در کش نمی‌ماند و دفعه‌ی بعد دوباره تلاش می‌شود.
- **نوشتن منتظر نمی‌ماند (`void`):** کاربر برای ذخیره‌شدن در Redis معطل نمی‌شود.

نحوه‌ی ذخیره‌سازی

- همه‌ی کلیدها پیشوند `paradiso:v1:` دارند. اگر شکل داده‌ی ذخیره‌شده عوض شد، `v1` را به `v2` تغییر دهید؛ کلیدهای قدیمی خودشان منقضی می‌شوند.
- مقدار به‌صورت JSON ذخیره می‌شود (پیشوند `j:`). اگر بزرگ‌تر از ۸ کیلوبایت باشد، با gzip فشرده و به‌صورت base۶۴ ذخیره می‌شود (پیشوند `z:`) تا حجم و زمان کمتر شود.
- مقدار خراب یا ناخوانا «Miss» حساب می‌شود، نه خطا.
- در لاگ خطاها، آدرس اتصال با `redis://***` پوشانده می‌شود و هر خطا حداکثر یک‌بار در دقیقه ثبت می‌شود.

۶. اتصال به `MovieDb.serviec.ts` و اثر آن

این فایل تنها دروازه‌ی پروژه به TMDb است؛ هر بخشی که فیلم یا بازیگر لازم دارد از این‌جا می‌گذرد. برای همین کش را همین‌جا گذاشتیم تا بقیه‌ی کد (کنترلرها و سرویس‌ها) بدون هیچ تغییری سریع‌تر شوند.

قبل و بعد

```
BEFORE:  caller -> get() -> fetch TMDb

AFTER:   caller -> get(endpoint, params, ttl)
           |-- ttl = 0 -> request() -> TMDb            (not cached)
           `-- ttl > 0 -> cache.remember(key, ttl, loader)
                          |-- in Redis -> return        (a few ms)
                          `-- miss -> request() -> TMDb -> save -> return
```

بخش‌های تغییر کرده

- **تزریق وابستگی:** سازنده حالا علاوه بر `ConfigService`، یک `CacheService` هم می‌گیرد.
- **`request()`:** همان `get()` قدیمی است که فقط اسمش عوض شد. منطق آن (کلید API، timeout پنج‌ثانیه‌ای، تلاش مجدد روی ۴۲۹، تبدیل خطاها) دست‌نخورده است و فقط همین متد با TMDb حرف می‌زند.
- **`get()` جدید:** پارامتر اختیاریِ `ttlSeconds` دارد. بدون آن مثل قبل رفتار می‌کند. با آن، یک کلید از «نام endpoint + پارامترهای مرتب‌شده» می‌سازد؛ زبان همیشه جزو کلید است و **کلید API هرگز در کلید نمی‌آید.**

- **امضای متدها عوض نشد:** `getMovieDetails` ، `getGenres` و بقیه همان ورودی و خروجی قبلی را دارند؛ پس هیچ فراخواننده‌ای نیاز به ویرایش نداشت.

مدت نگه‌داری هر نوع داده (TMDB_CACHE_TTL)

داده	کلید	مدت
ژانرها	`tmdb:genre/movie/list?...`	۳۰ روز
اطلاعات فیلم	`tmdb:movie/<id>?...`	۷ روز
پوسترها	`tmdb:movie/<id>/images?...`	۳۰ روز
بازیگر / کارگردان	`tmdb:person/<id>?...`	۳ روز
بازیگران محبوب	`tmdb:person/popular?...`	۲۴ ساعت
جستجوی فیلم (نتیجه‌ی فیلترشده)	`tmdb:search-movies:<page>:<query>`	۶ ساعت
جستجوی افراد	`tmdb:search/person?...`	۶ ساعت
حکم «کلیدواژه‌ی نامناسب» برای هر فیلم	`tmdb:erotic:<id>`	۷ روز

دو بخش خاص

۱) **`searchMovies`** : قبلاً یک جستجوی TMDb به‌اضافه‌ی حدود ۱۹ تماس کلیدواژه برای فیلتر نتایج داشت. حالا خودِ «نتیجه‌ی نهاییِ فیلترشده» کش می‌شود؛ یعنی یک Hit به‌جای حدود ۱۹ تماس فقط یک خواندن از Redis است. به همین دلیل جستجو بیشترین بهبود را دید.

۲) **`isErotic`** : حالا سه لایه دارد: اول نقشه‌ی داخل حافظه‌ی برنامه (لایه‌ی قبلی)، بعد Redis، و در آخر TMDb. با این کار بعد از ری‌استارت یا خواب‌رفتن سرور Render، حکم‌ها فراموش نمی‌شوند.

چه چیزی کش نمی‌شود؟

- `getMovieDetailsWithExtras` : فقط هنگام وارد کردن فیلم به کتابخانه استفاده می‌شود و حجم بسیار بزرگی دارد.
- خطاها، از جمله ۴۰۴ و timeout.
- **لیست مسدودی و فیلتر محتوای بزرگسال** بعد از کشِ TMDb و در کدِ فراخواننده اجرا می‌شوند. خودِ لیست مسدودی هم تا ۵ دقیقه کش می‌شود و اسکریپت ممیزی آن را پاک می‌کند؛ پس مسدودکردن یک فیلم بعد

از ممیزی فوراً، و در غیر این صورت حداکثر تا ۵ دقیقه بعد اثر می‌کند.

اثر کلی روی رفتار

جنبه	اثر
سرعت	تماس‌های تکراری بسیار سریع‌تر؛ درخواست‌های خیلی کند (۳ تا ۴ ثانیه) تقریباً حذف شدند
محدودیت TMDb	تعداد تماس‌ها به‌شدت کم می‌شود، پس خطای ۴۲۹ و مسیر تلاش مجدد کمتر پیش می‌آید
پس از ری‌استارت	کش از بین نمی‌رود (برخلاف نقشه‌های داخل حافظه)
خطا و قطعی Redis	برنامه مثل قبل عمل می‌کند
تازگی داده	اطلاعات فیلم تا ۷ روز، فرد تا ۳ روز و نتیجه‌ی جستجو تا ۶ ساعت می‌تواند قدیمی باشد
API و فرانت‌اند	هیچ تغییری ندارد؛ پاسخ‌ها یکسان‌اند

۷. نتیجه‌ی اندازه‌گیری روی Render

همان اسکریپت و همان روش (از طریق پروکسی، ۱۲ درخواست برای هر مسیر) قبل و بعد از استقرار اجرا شد. پروکسی حدود ۴۰۰ میلی‌ثانیه به همه‌ی اعداد اضافه می‌کند، پس فقط اجراهای هم‌روش را با هم مقایسه کنید. اعداد، میانه (median) بر حسب میلی‌ثانیه‌اند:

مسیر	قبل	بعد (کش خالی)	بعد (کش پر)
`/movies/tmdb/550`	۱۱۲۱	۹۶۲	**۸۵۳**
`/movies/tmdb/genres`	۱۰۳۷	۶۵۸	**۶۳۵**
جستجوی «batman»	۱۰۶۸	۸۵۴	**۸۷۹**
جستجوی ۱۲ عبارت متفاوت	≈ ۲۱۳۰	۲۰۷۰	**۸۷۶**
`/people/287`	۱۱۲۴	۱۱۷۱	۱۰۹۵ (بدون تغییر)

- **جستجو** بیشترین بهبود را داشت: از حدود ۲ ثانیه به حدود ۰٫۸۵ ثانیه.
- **Miss هزینه‌ی اضافه‌ای ندارد:** ستون «کش خالی» تقریباً با «قبل» برابر است.
- **مسیر افراد تغییر نکرد** چون زمانش بیشتر صرف دو پرس‌وجو از Neon می‌شود و از قبل هم یک کش ۲۴ ساعته در حافظه دارد.

- نمونه‌ها کم‌اند و از پروکسی عبور می‌کنند؛ الگو را ببینید، نه تفاوت‌های ۵۰ میلی‌ثانیه‌ای را.

فایل‌های نتیجه: `docs/perf/after-redis-cold.json` و `after-redis-warm.json`.

۸. نکات و محدودیت‌ها

- **منطقه‌ی Redis مهم است.** اگر Redis از Render دور باشد (مثلاً قاره‌ای دیگر)، هر خواندن از آن خودش صدها میلی‌ثانیه طول می‌کشد و ممکن است فایده‌ای نداشته باشد. دستوری که بیش از ۵۰۰ms طول بکشد Miss حساب می‌شود.
- **تغییر فیلتر محتوا:** اگر کد فیلتر محتوای بزرگسال را عوض کنید، جستجوها و حکم‌های ذخیره‌شده تا انقضا می‌مانند. پیشوند `v1` را عوض کنید تا همه پاک شوند.
- **Miss کمی بیشتر از قبل هزینه دارد:** یک رفت‌وبرگشت به Redis را هم می‌پردازد. اگر درخواست دوم قبل از تمام‌شدن نوشتنِ درخواست اول برسد، آن هم Miss می‌شود.
- **اشیای مشترک:** درخواست‌های هم‌زمان یک شیء را به اشتراک می‌گیرند؛ پس کدی که پاسخ را تغییر می‌دهد می‌تواند روی دیگری اثر بگذارد. در کد فعلی چنین موردی ندیدم.
- **محیط توسعه و تولید** ممکن است یک دیتابیس Redis مشترک داشته باشند. چون داده‌ی عمومی TMDb است مشکلی ندارد.

افزودن کش به یک متد جدید

```
// ttl is in seconds; pass it as the third argument to get()
getMovieCredits(movieId: number) {
  return this.get<Credits>(
    'movie/' + movieId + '/credits',
    {},
    TMDB_CACHE_TTL.movie,
  );
}
```

۹. ماژول سراسری: Redis چگونه Global تعریف شده است؟

مشکلی که ماژول سراسری حل می‌کند

در NestJS هر ماژول مثل یک «اتاق دربسته» است. سرویسی که در ماژول A تعریف شده، برای ماژول B دیده نمی‌شود، مگر این‌که ماژول A آن سرویس را `exports` کند **و** ماژول B هم ماژول A را در `imports` خودش بنویسد.

`MovieDbService` هم در `MoviesModule` و هم در `GenresModule` ثبت شده است و هر دو به کش نیاز دارند. اگر کش یک ماژول معمولی بود، باید در همه‌ی ماژول‌های مصرف‌کننده تکرار می‌کردیم: `imports: [CacheModule]`.

تعریف ماژول: فقط یک دکوراتور اضافه

```
// src/cache/cache.module.ts
import { Global, Module } from '@nestjs/common';
import { CacheService } from './cache.service';

// Global so any feature can inject CacheService without wiring.
@Global()
@Module({
  providers: [CacheService],
  exports: [CacheService],
})
export class CacheModule {}
```

- `@Global()` به Nest می‌گوید: «هر چه این ماژول export می‌کند را برای **همه** ماژول‌ها در دسترس بگذار.»
- `providers` : سرویسی که ماژول می‌سازد (`CacheService`).
- `exports` : آن چیزی که بیرون از ماژول دیده می‌شود. بدون این خط، حتی ماژول سراسری هم سرویس را پنهان نگه می‌دارد.

ثبت در ریشه: فقط یک بار

ماژول سراسری باید یک‌بار در جایی ثبت شود، معمولاً `AppModule` :

```
// src/app.module.ts
@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true, ... }),
    ScheduleModule.forRoot(),
    CacheModule,          // registered once, visible everywhere
    ...
  ],
})
export class AppModule {}
```

استفاده در هر سرویس، بدون `import` ماژول

```
// src/movies/providers/MovieDb.serviec.ts
@Injectable()
export class MovieDbService {
  constructor(
    private readonly configService: ConfigService,
    private readonly cache: CacheService,   // injected automatically
  ) {}
}
```

`MoviesModule` و `GenresModule` هیچ `imports: [CacheModule]` ای ندارند، ولی Nest `CacheService` را به `MovieDbService` در هر دو تزریق می‌کند.

چرا برای Redis درست است؟

- **زیرساخت مشترک است:** کش مثل تنظیمات یا لاگ، یک ابزار عمومی است و به یک «ویژگی» خاص تعلق ندارد. (پروژه همین الگو را برای تنظیمات هم دارد: `ConfigModule.forRoot({ isGlobal: true })`.)
- **یک اتصال برای کل برنامه:** سرویس‌های Nest به‌صورت پیش‌فرض singleton هستند؛ یعنی فقط یک نمونه‌ی `CacheService` و در نتیجه یک اتصال به Redis وجود دارد، حتی اگر ده ماژول از آن استفاده کنند. این برای پلن رایگان Upstash (محدودیت تعداد اتصال) مهم است.
- **بستن تمیز اتصال:** متد `onModuleDestroy` هنگام خاموش‌شدن برنامه اتصال را می‌بندد.
- **کدِ کمتر و کم‌خطاتر:** هر ماژول جدیدی که کش لازم دارد فقط `CacheService` را در سازنده می‌نویسد.

هشدار: Global را کم استفاده کنید

ماژول سراسری وابستگی‌ها را «پنهان» می‌کند: با نگاه به `imports` یک ماژول نمی‌شود فهمید چه چیزهایی از بیرون می‌گیرد. پس فقط برای زیرساخت واقعاً مشترک (کش، کانفیگ، لاگ) استفاده کنید، نه برای سرویس‌های کسب‌وکار مثل فیلم‌ها یا کاربران.

در تست‌ها

در تست واحد نیازی به ماژول Nest نیست؛ مستقیم ساخته می‌شود.

`new CacheService(new ConfigService({}))` یک کش «خاموش» می‌دهد که هر درخواست را Miss می‌کند، یعنی همان رفتار بدون Redis:

```
const noCache = () => new CacheService(new ConfigService({}));
const service = new MovieDbService(config, noCache());
```

خلاصه در یک نگاه

مورد	در این پروژه
نوع ماژول	سراسری (`@Global()`)
جای ثبت	یک‌بار در `AppModule`
چه چیزی export می‌شود	`CacheService`
چه کسانی مصرف می‌کنند	`MovieDbService` (در `MoviesModule` و `GenresModule`)، بدون import ماژول
تعداد اتصال Redis	یکی (singleton)
اگر Redis در دسترس نباشد	بدون خطا، فراخوان مستقیم به TMDb